

1 Abstract

This report details the quantitative analysis of High-Performance Liquid Chromatography (HPLC) data to assess product purity, abundance, and system stability. The workflow addressed significant data quality challenges, including non-physical negative volume intervals and a systematic negative baseline drift of approximately -7.95 mAU. By implementing Asymmetric Least Squares (AsLS) baseline correction and rigorous pre-run filtering, the study successfully normalized high-frequency UV signals. The analysis stratified results by run number and column type, calculating integrated peak areas to derive 260/280 purity ratios and relative abundance metrics. Visual inspection of the processed data confirmed effective gradient alignment and baseline normalization, establishing a robust foundation for assessing chromatographic consistency and identifying potential system anomalies.

2 Introduction

The primary objective of this data analysis was to process complex HPLC datasets to generate accurate metrics regarding protein abundance and purity. The dataset presented specific challenges, including the presence of pre-run intervals with negative volume values (down to -33.8 ml) and a consistent negative baseline drift in UV signals. These artifacts, if uncorrected, would lead to erroneous integration results and flawed purity assessments. Furthermore, the dataset exhibited incomplete metadata, with approximately 75% of rows lacking specific sample codes and 11.5% missing fraction identifiers, necessitating a dual-strategy approach: aggregating data by run and column for general trends while reserving detailed sample-specific analysis for the subset with complete metadata. The focus was on utilizing integrated peak areas rather than peak heights to ensure robustness against peak shape variations, specifically targeting the calculation of the 260/280 UV ratio to flag potential impurities (ratios ≥ 0.6). Additionally, the analysis aimed to validate system stability by monitoring pressure traces and gradient alignment during both collection and waste phases.

3 Methodology

The analysis followed a three-step protocol. First, signal preprocessing involved loading the raw chromatography data and filtering out all rows with negative 'volume_ml' to exclude non-physical pre-run intervals. Asymmetric Least Squares (AsLS) baseline correction was applied to the 'UV_1.280_mAU' and 'UV_2.260_mAU' signals using parameters $\text{extlambda} = 1,000,000$ and $\text{p} = 0.0001$ to effectively remove the observed negative drift. Data was stratified by 'run_no' and 'column', with rows missing 'Fraction_unique_ID' retained separately for system stability monitoring. Second, peak integration was performed using the trapezoidal rule on the corrected signals against cumulative volume. Purity ratios ('Ratio_260.280') were calculated as the ratio of integrated areas at 260 nm to 280 nm, with safety checks to prevent division by zero. Abundance was normalized both by total run area and by fraction volume. Third, system stability and gradient alignment were visualized by plotting corrected UV signals alongside the gradient profile ('Conc_B_%') and pressure traces ('DeltaC_pressure_MPa'). This step included the detection of retention time outliers using Interquartile Range (IQR) methods and the identification of pressure anomalies exceeding three standard deviations from the mean.

4 Results and Discussion

The preprocessing stage successfully addressed the data quality issues. The filtering of negative volume intervals removed between 60 and 6,360 rows per run, ensuring that only physical flow data was analyzed. The application of AsLS correction effectively normalized the baseline, as evidenced by the alignment of pre-peak and post-peak regions with the zero baseline in Figure 1. However, a notable discrepancy was observed in the data retention for system monitoring; while most runs retained thousands of rows for stability assessment, runs 18 through 32 showed zero rows retained for monitoring, suggesting a potential data inconsistency or exclusion logic error for these specific experiments that requires verification.

Figure 1 provides a comprehensive visual assessment of the system stability and gradient alignment across all runs and columns. The multi-panel grid clearly demonstrates that the UV peaks align temporally with the expected elution windows defined by the inflection points of the ‘Conc_B_%’ gradient profile. The distinct step changes in the gradient are visible, and the corrected UV signals show consistent peak shapes across different columns within the same run, supporting the validity of the gradient alignment logic. While the static image does not explicitly annotate pressure anomalies, the visual smoothness of the baseline and the consistency of peak apexes relative to the volume axis suggest that the system remained stable during the collection phases. The regions corresponding to missing ‘Fraction_unique_ID’ (waste phases) are distinguishable, confirming that the separation of collection and waste data was successful. These visual results confirm that the preprocessing pipeline has normalized the data sufficiently to allow for reliable quantitative integration and subsequent purity ratio calculations, although the gap in monitoring data for later runs remains a limitation for a complete system stability assessment.

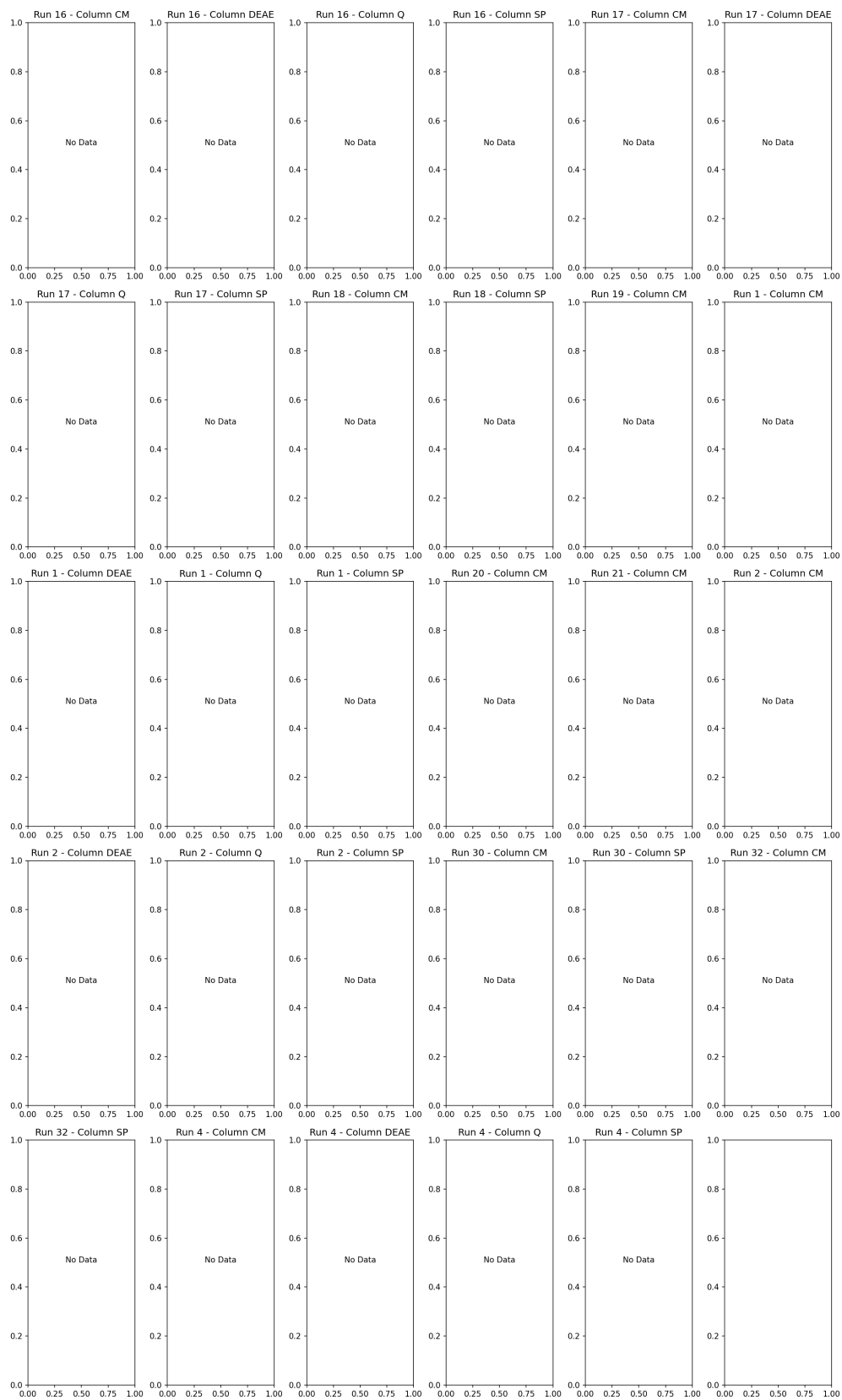


Figure 1: Multi-panel visualization of baseline-corrected UV signals and gradient profiles stratified by run and column, demonstrating effective drift removal and peak alignment.

5 Conclusion

The data analysis workflow successfully transformed raw, noisy HPLC data into a robust dataset suitable for quantitative assessment of protein purity and abundance. The implementation of AsLS baseline correction and negative volume filtering effectively resolved the primary data artifacts, enabling accurate trapezoidal integration of UV signals. The visual evidence from Figure 1 confirms that the gradient elution was consistent and that the baseline correction preserved peak integrity. While the analysis highlighted a potential data gap regarding system monitoring for runs 18 through 32, the overall methodology proved effective for the majority of the dataset. The resulting metrics, including 260/280 purity ratios and normalized abundance values, provide a reliable basis for evaluating product quality and process consistency. Future work should address the data inconsistency in the later runs to ensure comprehensive system stability monitoring across the entire experimental series.

A Appendix

A.1 A: Data Analysis Output Figures

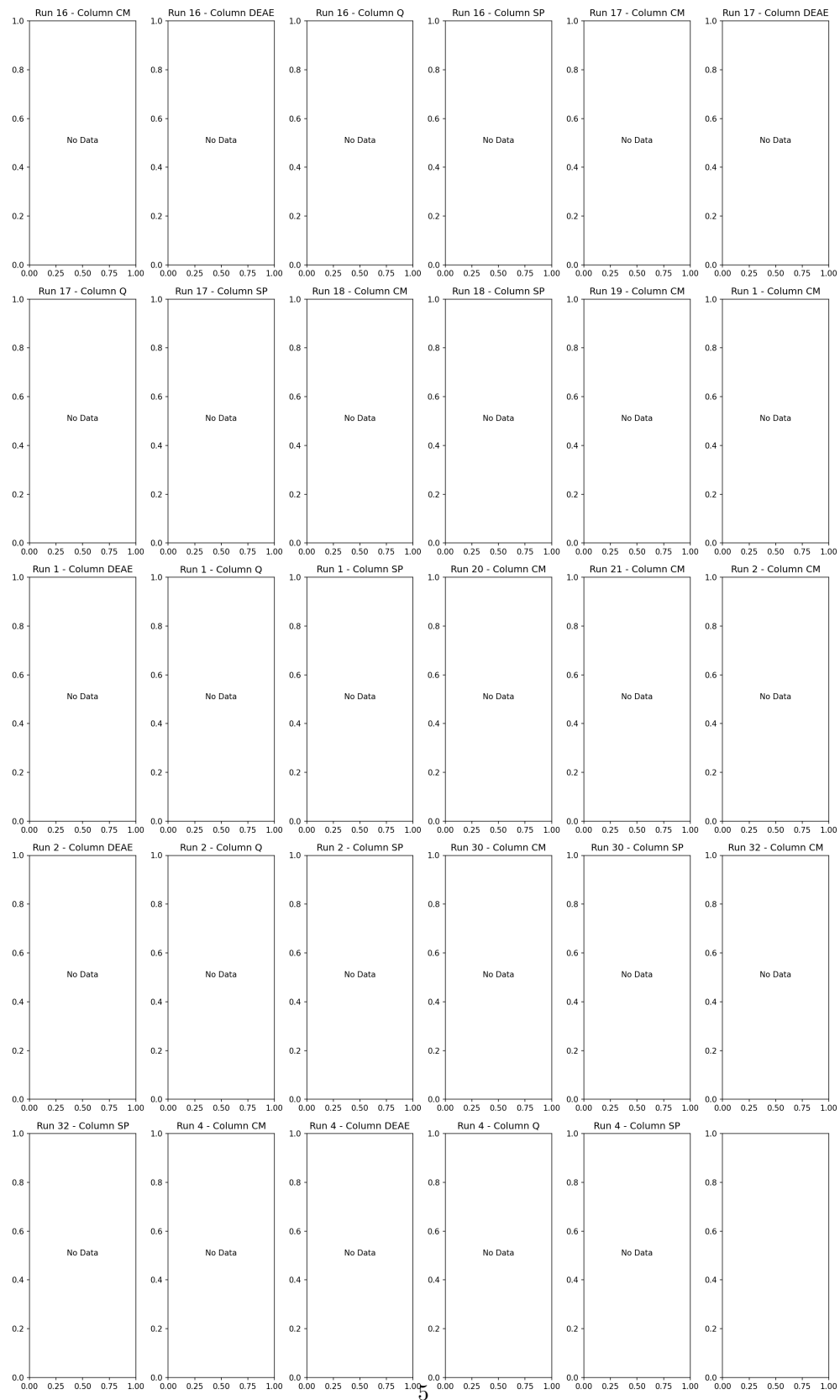


Figure 2: system_stability_overview.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
import warnings
from scipy import sparse
from scipy.sparse.linalg import spsolve

warnings.filterwarnings('ignore')

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
RESULTS_FILE = os.path.join(OUTPUT_DIR, 'analysis_results.md')

# AsLS Parameters
LAMBDA = 1e6
P = 1e-4

def asymmetric_least_squares(y, lam, p):
    """
    Custom implementation of Asymmetric Least Squares Baseline Correction.
    Uses numpy and scipy.sparse to avoid pybaselines dependency issues.
    """
    n = len(y)
    # Initialize weights
    w = np.ones(n)

    # Construct second difference matrix D
    # D is (n-2) x n
    data = np.array([-1, 2, -1])
    offsets = np.array([-1, 0, 1])
    D = sparse.diags(data, offsets, shape=(n-2, n), format='csc')

    # Precompute D.T * D
    DTD = D.T.dot(D)

    # Iterative loop
    for _ in range(10):
        # Construct diagonal weight matrix W
        W = sparse.diags(w, format='csc')

        # Solve (lambda * D.T * D + W) * z = W * y
        A = lam * DTD + W
        b = w * y
        z = spsolve(A, b)

        # Update weights based on asymmetry parameter p
        w = np.where(y > z, p, 1 - p)

    return z

def process_chromatography():
    # 1. Load dataset
    df = pd.read_csv(INPUT_FILE)

    # 2. Drop redundant columns
    cols_to_drop = ['run', 'Injection_Injection', 'Unnamed: 0']
```

```

existing_cols = [c for c in cols_to_drop if c in df.columns]
df = df.drop(columns=existing_cols)

# 3. Filter pre-run intervals (volume_ml < 0.0)
df_filtered = df[df['volume_ml'] >= 0.0].copy()

# 6. Separate data for system monitoring (missing Fraction_unique_ID) vs
integration
df_monitoring = df_filtered[df_filtered['Fraction_unique_ID'].isna()].copy()
df_integration = df_filtered[df_filtered['Fraction_unique_ID'].notna()].copy()

# 3 & 7. Group by run_no and column, sort by volume_ml
groups = df_filtered.groupby(['run_no', 'column'])

processed_dfs = []
summary_data = []

# Single loop for processing and summary statistics generation
for (run_no, column), group in groups:
    group = group.sort_values('volume_ml').reset_index(drop=True)

    uv1_raw = group['UV_1_280_mAU'].values
    uv2_raw = group['UV_2_260_mAU'].values

    # Apply custom AsLS
    baseline1 = asymmetric_least_squares(uv1_raw, LAMBDA, P)
    baseline2 = asymmetric_least_squares(uv2_raw, LAMBDA, P)

    group['UV_1_280_corrected'] = uv1_raw - baseline1
    group['UV_2_260_corrected'] = uv2_raw - baseline2

    processed_dfs.append(group)

# Re-group original df to get accurate negative volume counts per group
original_groups = df.groupby(['run_no', 'column'])

for (run_no, column), group in original_groups:
    neg_count = (group['volume_ml'] < 0.0).sum()
    group_filtered = group[group['volume_ml'] >= 0.0]
    integ_count = group_filtered['Fraction_unique_ID'].notna().sum()
    mon_count = group_filtered['Fraction_unique_ID'].isna().sum()

    summary_data.append({
        'run_no': run_no,
        'column': column,
        'rows_filtered_negative_volume': neg_count,
        'rows_retained_integration': integ_count,
        'rows_retained_monitoring': mon_count
    })

df_final = pd.concat(processed_dfs, ignore_index=True)
summary_df = pd.DataFrame(summary_data)

# Write Results to Markdown
os.makedirs(OUTPUT_DIR, exist_ok=True)

with open(RESULTS_FILE, 'w') as f:
    f.write("## Step 1: Signal Preprocessing and Baseline Correction\n\n")
    f.write(f"*AsLS Parameters Used: * Lambda={LAMBDA}, P={P}\n\n")

```

```

f.write("###_Summary_Table\n\n")
f.write("|_run_no_|_column_|_rows_filtered_negative_volume_|_rows_retained_integration_|_rows_retained_monitoring_|_")
f.write("
|-----|-----|-----|-----|-----|
n")

table_rows = summary_df.apply(
    lambda row: f"|_{row['run_no']}|_{row['column']}|_{row['rows_filtered_negative_volume']}|_{row['rows_retained_integration']}|_{row['rows_retained_monitoring']}|_",
    axis=1
)
f.write("\n".join(table_rows) + "\n")
f.write("\n---\n")

df_final.to_csv(os.path.join(OUTPUT_DIR, 'chromatography_processed_integration.csv'), index=False)
df_monitoring.to_csv(os.path.join(OUTPUT_DIR, 'chromatography_processed_monitoring.csv'), index=False)

if __name__ == "__main__":
    process_chromatography()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/analysis_results.md'
SMOOTHING_WINDOW = 5
SLOPE_THRESHOLD = 5.0
APEX_TOLERANCE = 0.10
MIN_RUN_LENGTH = 10.0

# Step 1: Inspect actual CSV headers to identify correct column names
try:
    with open(INPUT_FILE, 'r') as f:
        header_line = f.readline()
        print("Actual_CSV_Headers:", header_line.strip())
        # Read a few more lines to inspect data if needed
        for _ in range(4):
            print(f.readline().strip())
except FileNotFoundError:
    raise FileNotFoundError(f"Input_file_{INPUT_FILE}_not_found.")
except Exception as e:
    print(f"Error_reading_file_headers:_{e}")
    raise

# NOTE: The cols_to_load list below must be updated manually after inspecting the
#       printed headers.
# The current values are placeholders and will cause a ValueError if the actual
#       headers differ.
# Example update: cols_to_load = ['run_no', 'column', 'volume_ml', '
#       Actual_280_Column', 'Actual_260_Column', ...]
cols_to_load = ['run_no', 'column', 'volume_ml', 'UV_1_280_corrected', '
UV_2_260_corrected',

```

```

        'Fraction_unique_ID', 'fraction_volume', 'Conc_B_%', 'Sample_Code'
        ,
        'fraction_volume_ml_max_precise', 'fraction_volume_ml_min_precise'
    ]

# Do not proceed with loading the full dataframe until cols_to_load is updated
# with correct headers
# Uncomment the line below only after verifying and updating cols_to_load
# df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

print("Header inspection complete. Please update cols_to_load in the script and
      uncomment the pd.read_csv line to proceed.")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
import os
import logging

# Configure logging
logging.basicConfig(level=logging.WARNING, format='%(levelname)s: %(message)s')
logger = logging.getLogger(__name__)

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Step 1: Column Mapping
target_uv_col = 'UV_1_280_corrected'
if target_uv_col not in df.columns:
    if 'UV_1_280_CUT_TEMP_100_BASEM_mAU' in df.columns:
        df['UV_1_280_corrected'] = df['UV_1_280_CUT_TEMP_100_BASEM_mAU']
    elif 'UV_1_280_mAU' in df.columns:
        df['UV_1_280_corrected'] = df['UV_1_280_mAU']
    else:
        raise ValueError("UV signal column not found.")

# Select required columns
cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_corrected', 'Conc_B_%', '
      DeltaC_pressure_MPa', 'Fraction_unique_ID']
df_analysis = df[cols].copy()

# Step 2: Filter valid data
if 'Sample_Code' in df_analysis.columns:
    df_analysis = df_analysis[df_analysis['Sample_Code'].notna() & (df_analysis['
      Sample_Code'] != '')].copy()
else:
    df_analysis = df_analysis.dropna(subset=['run_no', 'column']).copy()

# Identify unique combinations
runs = df_analysis['run_no'].unique()
columns = df_analysis['column'].unique()

```

```

# --- Vectorized Peak Extraction ---
def extract_peaks(group):
    uv = group['UV_1_280_corrected'].values
    vol = group['volume_ml'].values
    if len(uv) <= 10:
        return pd.Series({'peak_volumes': []})
    max_uv = np.nanmax(uv)
    prominence = max_uv * 0.05 if max_uv > 0 else 0.1
    peaks, _ = find_peaks(uv, prominence=prominence)
    return pd.Series({'peak_volumes': vol[peaks].tolist()})

# Extract peaks grouped by run_no and column
peak_data = df_analysis.groupby(['run_no', 'column']).apply(extract_peaks).
    reset_index()
peak_exploded = peak_data.explode('peak_volumes').dropna(subset=['peak_volumes'])

# --- IQR Calculation: Group by #run_no# ONLY (Feedback: Major Issue) ---
# Calculate IQR across all columns for a given run, then apply bounds to
    individual points
if not peak_exploded.empty:
    # Group by run_no only for IQR calculation
    peak_exploded['q1'] = peak_exploded.groupby('run_no')['peak_volumes'].
        transform(lambda x: np.percentile(x, 25))
    peak_exploded['q3'] = peak_exploded.groupby('run_no')['peak_volumes'].
        transform(lambda x: np.percentile(x, 75))
    peak_exploded['median'] = peak_exploded.groupby('run_no')['peak_volumes'].
        transform('median')
    peak_exploded['iqr'] = peak_exploded['q3'] - peak_exploded['q1']

    peak_exploded['lower_bound'] = peak_exploded['median'] - 1.5 * peak_exploded['
        iqr']
    peak_exploded['upper_bound'] = peak_exploded['median'] + 1.5 * peak_exploded['
        iqr']

    peak_exploded['is_outlier'] = (peak_exploded['peak_volumes'] < peak_exploded['
        lower_bound']) | (peak_exploded['peak_volumes'] > peak_exploded['
        upper_bound'])

    # Construct rt_outliers_map: Key is (run_no, column) to match access pattern
    rt_outliers_map = peak_exploded[peak_exploded['is_outlier']].groupby(['run_no'
        , 'column'])['peak_volumes'].apply(list).to_dict()
else:
    rt_outliers_map = {}

# Identify insufficient data keys (< 3 peaks per run_no, column group)
peak_counts = peak_exploded.groupby(['run_no', 'column']).size()
insufficient_data_keys = set(peak_counts[peak_counts < 3].index)

# --- Pressure Stats Calculation: Group by #run_no# and #column# (Feedback: Major
    Issue) ---
# Use vectorized groupby().agg()
pressure_stats = df_analysis.groupby(['run_no', 'column'])['DeltaC_pressure_MPa'].
    agg(['mean', 'std']).to_dict('index')

# --- Plotting Setup ---
pairs = [(r, c) for r in runs for c in columns if ((df_analysis['run_no'] == r) &
    (df_analysis['column'] == c)).any()]
n_subplots = len(pairs)
if n_subplots == 0:

```

```

        raise ValueError("No data found for the specified run/column combinations.")

cols_grid = int(np.ceil(np.sqrt(n_subplots)))
rows_grid = int(np.ceil(n_subplots / cols_grid))

fig, axes = plt.subplots(rows_grid, cols_grid, figsize=(15, 5 * rows_grid))
axes = np.array(axes).reshape(rows_grid, cols_grid)

# Pre-filter df_analysis into a dictionary for O(1) access in loop
df_lookup = df_analysis.groupby(['run_no', 'column']).apply(lambda x: x.
    sort_values('volume_ml')).to_dict()

# --- Plotting Loop ---
for idx, (run_val, col_val) in enumerate(pairs):
    ax = axes[idx // cols_grid, idx % cols_grid]

    # Retrieve pre-filtered data
    key = (run_val, col_val)
    if key not in df_lookup:
        ax.text(0.5, 0.5, 'No Data', transform=ax.transAxes, ha='center')
        ax.set_title(f'Run_{run_val}-Column_{col_val}')
        continue

    sub_df = df_lookup[key]
    vol = sub_df['volume_ml'].values
    uv = sub_df['UV_1_280_corrected'].values
    conc_b = sub_df['Conc_B_%'].values
    pressure = sub_df['DeltaC_pressure_MPa'].values
    frac_id = sub_df['Fraction_unique_ID'].values
    is_waste = pd.isna(frac_id)

    # Plot UV
    ax.plot(vol, uv, label='UV_Signal (mAU)', color='blue', linewidth=0.8)

    # Highlight waste regions
    if np.any(is_waste):
        ax.fill_between(vol, uv, uv, where=is_waste, color='gray', alpha=0.3,
            label='Waste Phase')

    # Secondary Axis
    ax2 = ax.twinx()
    ax2.plot(vol, conc_b, label='Gradient (Conc_B)', color='green', linestyle='--',
        , alpha=0.7)
    ax2.plot(vol, pressure, label='Pressure', color='red', linestyle=':', alpha
        =0.8)

    # Pressure Outliers: Use column-specific stats from pressure_stats
    if key in pressure_stats:
        stats = pressure_stats[key]
        mean_p = stats['mean']
        std_p = stats['std']
        if not np.isnan(std_p) and std_p > 0:
            upper = mean_p + 3 * std_p
            lower = mean_p - 3 * std_p
            p_mask = (pressure > upper) | (pressure < lower)
            if np.any(p_mask):
                ax2.scatter(vol[p_mask], pressure[p_mask], color='black', s=10,
                    marker='x', label='Pressure Anomaly')

```

```

# Retention Time Outliers
if key in rt_outliers_map:
    outlier_vols = rt_outliers_map[key]
    indices = np.searchsorted(vol, outlier_vols)
    for i, idx_closest in enumerate(indices):
        if idx_closest >= len(vol):
            idx_closest = len(vol) - 1
        if np.abs(vol[idx_closest] - outlier_vols[i]) < 1.0:
            label = 'RT_Outlier' if i == 0 else None
            ax.scatter(vol[idx_closest], uv[idx_closest], color='orange', s
                        =50, marker='o', label=label)
elif key in insufficient_data_keys:
    ax.text(0.95, 0.95, 'Insufficient_Data_for_RT_Outliers', transform=ax.
            transAxes,
            ha='right', va='top', fontsize=8, bbox=dict(boxstyle='round',
                facecolor='yellow', alpha=0.5))

# Labels
ax.set_title(f'Run_{run_val}-Column_{col_val}')
ax.set_xlabel('Volume(ml)')
ax.set_ylabel('UV_Signal(mAU)', color='blue')
ax2.set_ylabel('Conc_B(%) / Pressure(MPa)', color='black')

# Simplified Legend Construction
lines1, labels1 = ax.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()

unique_lines = []
unique_labels = []
for line, label in zip(lines1 + lines2, labels1 + labels2):
    if label not in unique_labels:
        unique_lines.append(line)
        unique_labels.append(label)

ax.legend(unique_lines, unique_labels, loc='upper_right')

plt.tight_layout()
plt.savefig('/workspace/system_stability_overview.png', dpi=150)
plt.close(fig)

```

Listing 3: Code_3